SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA0620	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3: Articulation** (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	A . DEKKSHITHA	**Reg. No.:**	192572133
Section / Batch:		**Date:**	06.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Q1. Merge Sort

Analyze the scalability of Merge Sort by applying it to datasets of increasing sizes (e.g., 10^3, 10^4, 10^5 elements). Record execution time and plot the growth trend. Interpret the results using asymptotic analysis and explain how the divide and conquer approach ensures predictable performance. Justify its suitability for large-scale data processing.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CODE :

```
import random
import time
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
sizes = [1000, 10000, 100000]
print("Merge Sort Performance Analysis")
print("-" * 40)
for size in sizes:
    arr = [random.randint(1, 1000000) for _ in range(size)]
    start = time.perf_counter()
    merge_sort(arr)
    end = time.perf_counter()
    print("Dataset Size:", size)
    print("Execution Time:", round(end - start, 6), "seconds")
    print()
```

OUTPUT :

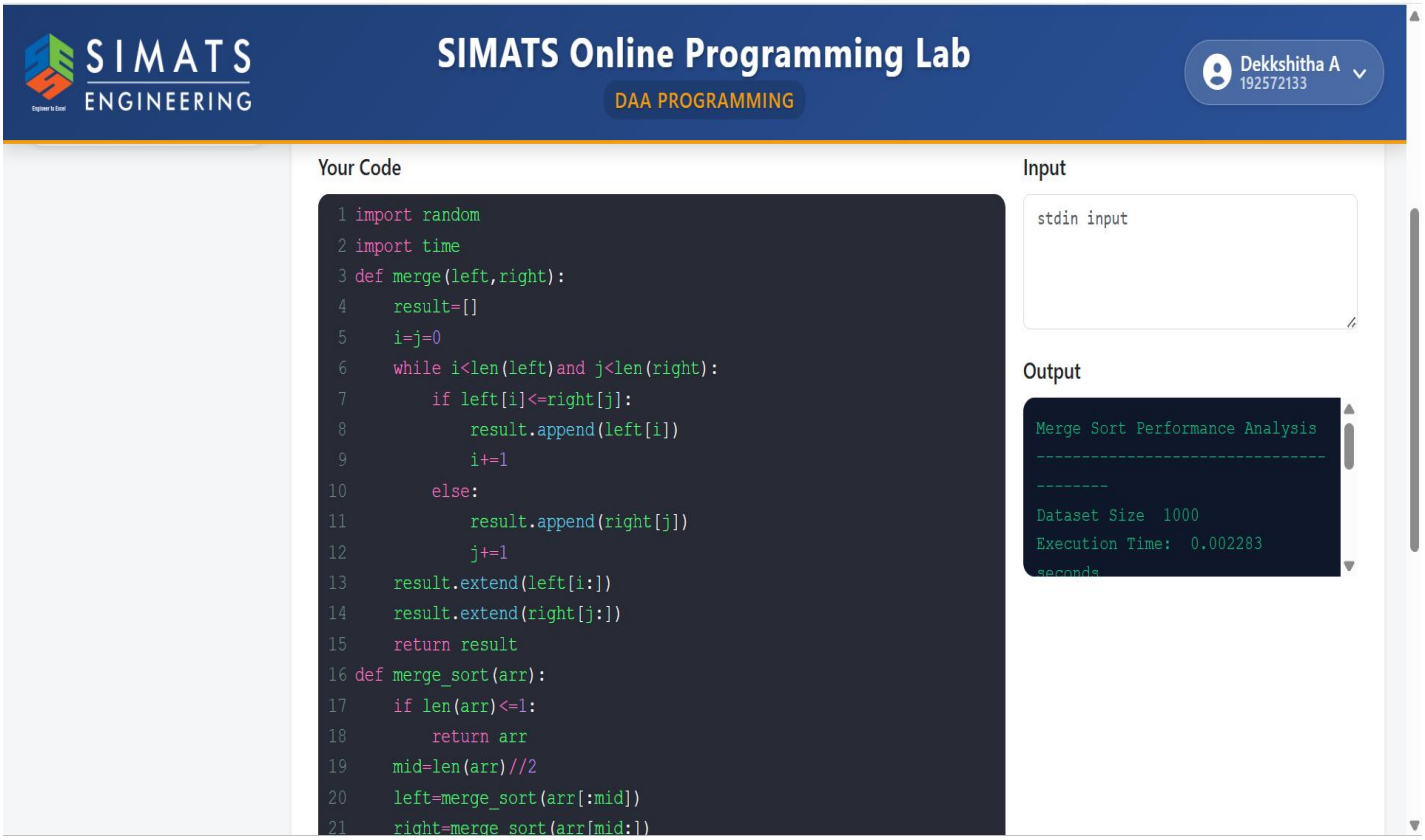
Merge Sort Performance Analysis

Dataset Size: 1000
Execution Time: 0.001815 seconds

Dataset Size: 10000
Execution Time: 0.023248 seconds

Dataset Size: 100000
Execution Time: 0.292911 seconds

EXECUTION :



```
    return result
def merge_sort(arr):
    if len(arr)<=1:
        return arr
    mid=len(arr)//2
    left=merge_sort(arr[:mid])
    right=merge_sort(arr[mid:])
    return merge(left,right)
sizes=[1000,10000,100000]
print("Merge Sort Performance Analysis")
print("-"*40)
for size in sizes:
    arr=[random.randint(1,1000000)for _ in range(size)]
    start=time.perf_counter()
    merge_sort(arr)
    end=time.perf_counter()
    print(f"Dataset Size ", size)
    print(f"Execution Time: ",round(end - start,6), "seconds")
    print()
```

Submitted